API Documentation

This document gives you a clear overview of how to work with the portal's APIs. The goal is to help developers understand the available endpoints, how to structure requests, what responses look like, and how to handle errors without digging through code.

---

1. Overview
The API lets external systems interact with the portal by fetching data, submitting updates, and triggering specific actions. All requests follow standard REST conventions. Every endpoint uses predictable patterns so developers can slot new calls into existing integrations with minimal effort.

2. Authentication
Most routes require a valid token. Clients usually authenticate through a login endpoint that returns a short-lived access token. Include this token in the header for every protected request. If the token expires, refresh it or request a new one. When authentication fails, the API responds with a status code that clearly indicates what went wrong.

3. Request structure
Always send data in a simple JSON format. Keep fields clean and match the naming convention used across the platform. If an endpoint needs optional parameters, include only what you need. This prevents conflicts and makes debugging easier.

4. Response structure
Every response returns a predictable pattern:
• a status indicator showing whether the request succeeded
• a message that explains the result in plain language
• a data field that contains the actual response payload
When something goes wrong, the API still uses the same structure so errors are easy to parse programmatically.

5. Common endpoints
You'll typically work with routes that handle user data, session management, content retrieval, and accessory information. Each endpoint follows a consistent naming style and versioning approach. For example, read-only routes rely on standard GET requests, update routes use POST or PATCH, and destructive actions require DELETE. Keep calls focused and avoid mixing multiple actions in a single request.

6. Versioning
The API uses versioned routes to avoid breaking older integrations. When a major change rolls out, a new version is introduced. Existing clients can continue using older

routes until they transition. This avoids sudden failures and gives teams enough time to adjust their workflows.

7. Error handling
Most failures fall into a few categories: missing authentication, invalid input, unavailable resources, or internal processing errors. The API returns clear error messages so you can identify the cause quickly. If the issue is client-side, fix the request and retry. If the server returns a temporary error, wait a moment and try again.

8. Rate limits
To keep the system stable, the API enforces basic rate limits. If a client makes too many requests too quickly, it receives a throttling response. Back off briefly before sending more requests. This avoids blocking and keeps integrations running smoothly.

9. Best practices
Use consistent headers, validate input before sending it, and avoid large unnecessary payloads. Log responses during development so issues surface early. When deploying to production, rely on stable versioned endpoints instead of experimental routes.